

Components of a Use Case in Software Engineering

1 Introduction

Every use case is composed of several essential elements that describe how a system interacts with users to achieve specific goals. By defining these components clearly, analysts ensure that use cases are **complete, traceable, and easy to understand** for both technical and non-technical stakeholders.

Definition

A **use case component** is a structural or descriptive element that defines a particular aspect of the user–system interaction, such as actors, preconditions, main flows, or exceptions.

2 Core Components of a Use Case

A well-written use case typically includes the following key components:

2.1 1. Use Case Name

Purpose

Provides a clear, concise, and action-oriented title describing the user’s goal.

Guidelines:

- Use a **verb–noun** format (e.g., “Login to System”, “Process Payment”).
- Should directly indicate the primary purpose of the use case.

Example: UC01 – Place Online Order

2.2 2. Actors

Actors represent the roles interacting with the system to achieve goals.

Definition

An **actor** is an external entity (person, device, or system) that interacts with the system to achieve a specific objective or is affected by the system's behavior.

Types of Actors:

- **Primary Actor:** Initiates the use case to achieve a goal. This actor directly benefits from the system's function.
- **Secondary Actor (Supporting Actor):** Assists the system in carrying out the use case by providing services or resources.
- **Offstage Actor (Tertiary Actor):** Does not directly interact with the system but is influenced by the outcome of the use case or has an interest in its results.

Explanation

Offstage actors are often stakeholders or external systems that observe or depend on the results of a use case but do not participate actively in it. For example, a government agency that receives automated compliance reports after a transaction is completed.

Example:

- **Primary Actor:** Customer – initiates order placement.
- **Secondary Actor:** Payment Gateway – verifies and processes payments.
- **Offstage Actor:** Tax Authority – receives sales report generated after the transaction.

2.3 3. Goal / Brief Description

This section summarizes the purpose or intention behind the use case.

Example: *“The customer places an order and receives a confirmation after successful payment.”*

Tip

Keep the goal statement short (2–3 lines) but ensure it expresses the core functionality.

2.4 4. Preconditions

Conditions that must be true before the use case begins.

Purpose

Preconditions ensure the system and actor are in valid states for interaction.

Example:

- User must be logged in.
- Items must exist in the shopping cart.

2.5 5. Postconditions

Defines the state of the system after the use case finishes successfully.

Example:

- Order is recorded in the database.
- Payment confirmation email is sent.

2.6 6. Trigger

Specifies the event that initiates the use case.

Examples:

- User clicks “Checkout”.
- Scheduled system event starts batch processing.

2.7 7. Main Flow (Basic Path)

Describes the primary steps that lead to successful completion of the use case.

Purpose

The main flow captures the **normal scenario**—the ideal sequence of interactions between actor and system.

Example – “Place Online Order”

1. Customer selects items and proceeds to checkout.
2. System displays order summary.
3. Customer enters delivery and payment information.
4. System verifies payment.
5. System generates an order ID and displays confirmation.

2.8 8. Alternate Flows (Extensions)

Captures alternative paths or exceptions that deviate from the main flow.

Purpose

Alternate flows handle exceptions, errors, or optional interactions while maintaining logical consistency.

Example – Payment Failure Path:

1. If payment fails, the system notifies the user and requests a retry.
2. User selects a different payment method.
3. System reprocesses payment.

2.9 9. Exceptions

Handles abnormal conditions or system errors that prevent successful completion.

Example:

- Network failure during transaction.
- Invalid user credentials.

2.10 10. Relationships

Defines connections between use cases or between actors and use cases.

Types:

- **Include:** Common functionality reused by multiple use cases.
- **Extend:** Optional or conditional behavior added to a base use case.
- **Generalization:** Inheritance between actors or use cases.

Example:

- “*Validate Payment*” is included in “*Process Online Order*”.

3 Example Template for Writing Use Cases

Use Case Name	Place Online Order
Actor(s)	Customer, Payment Gateway
Preconditions	Customer logged in, items in cart
Trigger	Click “Checkout”
Main Flow	1. Customer reviews order 2. System displays summary 3. Payment is processed 4. System confirms order
Alternate Flow	Payment failure leads to retry option
Postconditions	Order recorded, email sent

4 Use Case Section Purpose and Guidelines

Each section of a use case serves a distinct purpose to ensure clarity, completeness, and traceability.

Section	Purpose / Guidelines
Use Case Name	Start with a verb to describe an action clearly.
Scope	Defines the system or subsystem under consideration.
Level	Indicates the abstraction level: <i>summary goal</i> , <i>user goal</i> , or <i>subfunction</i> .
Primary Actor	The main entity that calls upon the system to deliver its service.
Stakeholders and Interests	Identifies who cares about this use case and what they want from it.
Preconditions	Specifies what must be true before the use case starts, and is worth noting to the reader.
Minimal Guarantees	States the fewest promises the system makes to stakeholders even if the use case fails.
Success Guarantee	Defines what must be true after successful completion of the use case.
Main Success Scenario	Describes the typical sequence of interactions that leads to success.
Extensions	Lists alternate scenarios for both success and failure.
Special Requirements	Specifies related non-functional or quality requirements (e.g., performance, usability).
Technology and Data Variation List	Describes variations in input/output devices, data formats, or protocols.
Frequency of Occurrence	Indicates how often the use case occurs; helps plan testing and prioritization.
Miscellaneous	Notes open issues, constraints, or additional observations.

5 Summary

Each component of a use case plays a critical role in defining the complete interaction between user and system. A well-structured use case with clear preconditions, flows, and postconditions ensures:

- Accurate requirement understanding.

- Reduced ambiguity for developers and testers.
- Easier validation of system behavior.

In essence, these components transform stakeholder goals into precise, verifiable software interactions.